

252-0027

Introduction to Programming Exercises

Week 3: Introduction to Java

First name Last name

**Department of Computer Science
ETH Zurich**

Organization

My name: *First name Last name*

For questions: *ETH email*

Please put «[EProg26]» in the subject

New exercises: **Tuesday evening** (normally)

Submit exercises by **Tuesday evening (23:59)** following week

Always submit via Git

Solutions in a separate Git project

Theory

Evaluation – Arithmetic Operators

Operator	Beschreibung	Kurzbeispiel
+	Addition	<code>int antwort = 40 + 2;</code>
-	Subtraktion	<code>int antwort = 48 - 6;</code>
*	Multiplikation	<code>int antwort = 2 * 21;</code>
/	Division	<code>int antwort = 84 / 2;</code>
%	Teilerrest, Modulo-Operation, errechnet den Rest einer Division	<code>int antwort = 99 % 57;</code>
+	positives Vorzeichen	<code>int j = +3;</code>
-	negatives Vorzeichen	<code>int minusJ = -j;</code>

Associativity

The associativity of an operator determines how an operand is used

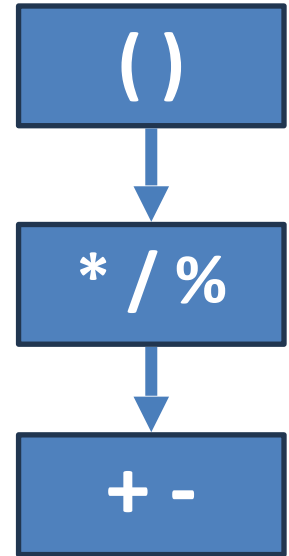
If an operator is $\otimes$

Left-associative $\rightarrow X \otimes Y \otimes Z$ equals $(X \otimes Y) \otimes Z$

Right-associative $\rightarrow X \otimes Y \otimes Z$ equals $X \otimes (Y \otimes Z)$

Evaluation – Order

- Operand is used by the operator with higher precedence
- If two operators have the same precedence, associativity decides
- If something else is desired: use parentheses!



Evaluation – Examples

()

* / %

+ -

```
1 public class Main {  
2     public static void main(String[] args) {  
3         System.out.println(2/3+4.5/3);  
4     }  
5 }
```

Output: 1.5

Evaluation – Examples

()

* / %

+ -

```
1 public class Main {  
2     public static void main(String[] args) {  
3         System.out.println(8%3+2.5+4/3);  
4     }  
5 }
```

Output: 5.5

Evaluation – Casting

()

* / %

+ -

```
1 public class Main {  
2     public static void main(String[] args) {  
3         System.out.println((double)5/2);  
4     }  
5 }
```

Output: 2.5

Evaluation – String

()

* / %

+ -

```
1 public class Main {  
2     public static void main(String[] args) {  
3         System.out.println("Wir schreiben das Jahr "+2023/2*2.0);  
4     }  
5 }  
6
```

Output: We are writing the year 2022.0

Evaluation – Exam Example

()

* / %

+ -

```
1 public class Main {  
2     public static void main(String[] args) {  
3         System.out.println(12/4*3+("X"+7%3)+18%5*2);  
4     }  
5 }
```

Output: 9X16

Evaluation – Exam Example

()

* / %

+ -

```
1
2 public class Main {
3     public static void main(String[] args) {
4         System.out.println("Wir schreiben das Jahr " + 2000 + 24);
5     }
6 }
```

Output: We are writing the year 200024

Exercise 1

```
1  class MyClass{  
2      public static void main(String[] args){  
3          System.out.println(9 / 3 * 2 + 1 + "" + 2 * 3 + (4 * 3) % 3);  
4      }  
5  }
```

760

Exercise 2



```
1  class MyClass{
2      public static void main(String[] args){
3          System.out.println(8 / 5 + 0.5 + 5 / 2 + (8 % 3) * 1.0);
4      }
5  }
```

5.5

Short-circuit evaluation («short-circuit evaluation»)

When computing `p && q` and `p || q`

Java first evaluates the left operand (`p`) and then the right one (`q`)

evaluation stops as soon as the result is fixed.

For `p && q`: if `p == false`, then `p && q == false`

For `p || q`: if `p == true`, then `p || q == true`

p	q	p && q
true	true	true
true	false	false
false	true	false
false	false	false

p	q	p q
true	true	true
true	false	true
false	true	true
false	false	false

In these cases, evaluating `q` is not needed!

Kahoot

Review Exercise 2

Any further questions about IntelliJ or Git?

(Interactive)

Solution Exercise 2, Exercise 2

```
/*  
 * Author: Maximiliana Muster  
 * für Einführung in die Programmierung  
 */  
public class HelloProgrammer {  
  
    public static void main(String[] args) {  
        System.out.println("Hello, my name is Max!");  
    }  
}
```

Solution Exercise 2, Exercise 4

1. Erstellen Sie eine Beschreibung `<geradezahl>`, die als legale Symbole alle geraden Zahlen (d.h. Zahlen, die ohne Rest durch 2 teilbar sind) zulässt. Beispiele sind +02, 4, 10, -20.

Solution Exercise 2, Exercise 4

2. Zeigen Sie in einer Tabelle, dass Ihre Beschreibung das Symbol “28” als gerade Zahl erkennt.

Solution Exercise 2, Exercise 4

- Erstellen Sie eine Beschreibung `<x2ygemischt>`, die als legale Symbole genau jene Wörter zulässt, in denen für jedes "X" zwei "Y" als Paar auftreten. Beispiele sind `XYX`, `YYX`, `XYYYXX`, `XXYYYY`. Die Beschreibung schliesst den leeren String nicht aus.

Solution Exercise 2, Exercise 4

Die folgenden EBNF-Beschreibungen sind nicht äquivalent. Finden Sie ein *kürzestmögliches* Symbol, das von der einen Beschreibung als legal erkannt wird, aber nicht von der anderen. (Fangen Sie mit einfachen Kombinationen von A und B an.)

EBNF-Beschreibung: <beispiel1>

<beispiel1> $\Leftarrow$ [A][B]

EBNF-Beschreibung: <beispiel2>

<beispiel2> $\Leftarrow$ [A[B]]

Solution Exercise 2, Exercise 4

5. Erstellen Sie eine EBNF Beschreibung `<doppelt>`, die als legale Symbole genau jene Wörter zulässt, in denen die doppelte Anzahl "Y" nach einer Folge von "X" auftritt. Beispiele sind `XYX`, `XXYYYY`, usw. Die Beschreibung schliesst den leeren String nicht aus.

Preview Exercise 3

Exercise 1: Debugging

In diesem Projekt befindet sich das fehlerhafte Programm `FollerVehler.java` im separaten Ordner `broken`. Dieser ist im Projekt als *Sources* markiert, aber zusätzlich *Excluded*, damit fehlerhafte Dateien beim Start das Projekt nicht blockieren. Finden und beheben Sie alle Fehler im Programm. IntelliJ zeigt an, wo Fehler auftreten, doch Sie müssen selbst herausfinden, was das Problem ist und wie es behoben werden kann. Nach erfolgreicher Korrektur sollte das Programm folgendes ausgeben:

```
Hello world
```

```
Gefällt Ihnen dieses Programm?
```

```
Ich habe es selbst geschrieben.
```

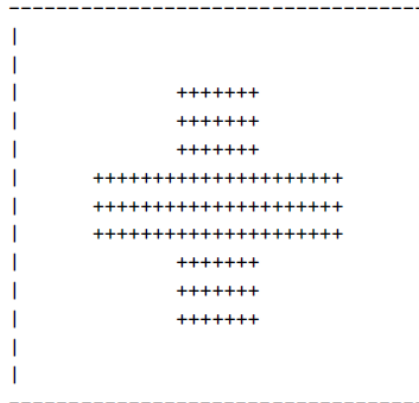
Exercise 1: Debugging

- In Java, all Java files in a project are compiled.
- If even one Java file does not compile, no Java code can be executed in that project.
- Therefore, we use subprojects so that exercises 2-4 compile even if Exercise 1 does not compile.

Exercise 2: Swiss Flag

Statt eine Vorlage zu benützen, schreiben Sie in dieser Aufgabe ein Programm von Grund auf selbst. IntelliJ wird Ihnen allerdings etwas Schreibarbeit abnehmen.

1. Erstellen Sie eine neue Java-Datei "SwissFlag.java". Rechts-Klicken Sie dafür den *src*-Ordner und wählen Sie *New* → *Java Class*. Geben Sie den Namen der Klasse ein ("SwissFlag.java") und bestätigen Sie mit der Enter-Taste. Danach wählen Sie *Add*, wenn Sie gefragt werden, ob Sie die Datei zu git hinzufügen möchten.
2. Erstellen Sie zuerst eine leere *main*-Methode, so wie Sie es bei anderen Programmen gesehen haben.
3. Erweitern Sie das Programm so, dass es die Schweizerfahne in der Konsole ausgibt. Die Fahne könnte ungefähr wie folgt aussehen, Sie dürfen aber auch eine grössere oder schönere Version entwerfen:



Teilen Sie das Programm in mehrere Methoden auf, welche von der *main*-Methode aufgerufen werden. Damit sorgen Sie dafür, dass weniger Wiederholungen von Code-Stücken vorkommen, was das Ändern des Programms deutlich einfacher macht.

Exercise 3: Input and Randomness

Aufgabe 3: Eingabe und Zufall

In dieser Aufgabe arbeiten Sie mit der Eingabe und Ausgabe von Java und lernen die Klassen `Scanner` und `Random` näher kennen.

1. Schreiben Sie ein Programm "Adder.java", welches zwei ganze Zahlen einliest und die Summe davon ausgibt. Sie sollen dafür die `Scanner`-Klasse verwenden, wie in der Vorlesung gezeigt. Das Programm soll nach der ersten Zahl fragen:

Geben Sie Zahl 1 ein:

dann nach der zweiten Zahl:

Geben Sie Zahl 2 ein:

und schliesslich, wenn Sie zum Beispiel "4" und "1999" eingeben, folgendes ausgeben:

4 + 1999 = 2003

Sie können davon ausgehen, dass nur ganze Zahlen eingegeben werden. Testen Sie das Programm mit verschiedenen Zahlen.

2. Schreiben Sie ein Programm `Wuerfel.java`, das einen Würfel simuliert. Hierbei soll eine positive ganze Zahl `N` eingelesen werden, welche die Anzahl der Seiten des Würfels repräsentiert. Der übliche Würfel hat 6 Seiten, jedoch existieren auch Würfel mit 12, 16, 20 (siehe Abbildung 1) und mehr Seiten. Jede Seite trägt eine unterschiedliche Zahl, die von 1 bis `N` (inklusive `N`) reicht.

Das Programm soll den Wurf simulieren, indem es die Klasse `Random` verwendet. Ein möglicher Ablauf des Programmes könnte folgendermassen aussehen:

Wie viele Seiten hat Ihr Würfel?

Der Benutzer gibt eine Zahl ein, z.B. 20, danach wird gewürfelt:

Es wurde eine 17 gewürfelt!



3. Schreiben Sie ein Programm `ChatGTP.java`, welches die Interaktion mit einem AI-Chatbot simuliert. Das Programm soll den Benutzer begrüßen, Sie nach Ihrem Namen und Alter fragen, und dem Benutzer Ihre Glückszahl verraten. Die Glückszahl soll zufällig gewählt werden, in dem Sie die `Random` Klasse benutzen. Ein möglicher Ablauf des Programmes könnte folgendermassen aussehen:

Guten Tag! Ich bin ChatGTP, der beste Chatbot, denn es gibt! Wie heissen Sie?

Ein Namen wird eingegeben, z.B. Alice, danach antwortet der Chatbot auf diesen Input:

Sehr erfreut Alice! Wie alt sind Sie?

Ein Alter wird eingegeben, z.B. 23, danach antwortet der Chatbot auf diesen Input:

Mittels dieser Informationen habe ich Ihre Glückszahl gefunden! Die Glückszahl lautet 42.

Exercise 4: Calculation

2. Vervollständigen Sie “SharedDigit.java”. In der Main-Methode sind zwei int Variablen a und b deklariert und mit einem Wert zwischen 10 und 99 (einschliesslich) initialisiert. Das Programm soll einer int Variablen r einen bestimmten Wert zuweisen. Wenn a und b eine Ziffer gemeinsam haben, dann wird r die gemeinsame Ziffer zugewiesen (wenn a und b beide Ziffern gemeinsam haben, dann kann eine beliebige Ziffer zugewiesen werden). Wenn es keine gemeinsame Ziffer gibt, dann soll -1 zugewiesen. Sie brauchen für dieses Programm keine Schleife.

Beispiele:

- Wenn a: 34 und b: 53, dann ist r: 3
- Wenn a: 10 und b: 22, dann ist r: -1
- Wenn a: 66 und b: 66, dann ist r: 6
- Wenn a: 34 und b: 34, dann ist r: 3 oder 4

Testen Sie Ihre Loesung mit a gleich 34 und b gleich 43. Was liefert Ihr Programm?

2. Vervollständigen Sie "SumPattern.java". In der Main-Methode sind drei int Variablen a, b, und c deklariert und mit irgendwelchen Werten initialisiert. Wenn die Summe von zwei der Variablen die dritte ergibt, nehmen wir an dass $a + c == b$, so soll die Methode "Moeglich. $a + c == b$ " ausgeben (wobei die Werte für a, b, und c einzusetzen sind). Wenn das nicht der Fall ist, dann soll die Methode "Unmoeglich." ausgeben.

Beispiele:

- Wenn a: 4, b: 10, c: 6, dann wird "Moeglich. $4 + 6 == 10$ " oder "Moeglich. $6 + 4 == 10$ " ausgegeben.
- Wenn a: 2, b: 12, c: 0, dann wird "Unmoeglich." ausgegeben.

3. Vervollständigen Sie "AbsoluteMax.java". In der Main-Methode sind drei int Variablen a, b, und c deklariert und mit irgendwelchen Werten initialisiert. Das Programm soll einer int Variable r den grössten absoluten Wert von a, b, und c zuweisen.

Exercise 5: EBNF

Aufgabe 5: EBNF

In dieser Aufgabe erstellen Sie erneut verschiedene EBNF-Beschreibungen. Speichern Sie diese wie gewohnt in der Text-Datei "EBNF.txt", welche sich in Ihrem "u02"-Ordner bzw. im "U02 <N-ETHZ-Account>"-Projekt befindet. Sie können die Datei direkt in Eclipse bearbeiten.

1. Erstellen Sie eine Beschreibung <pyramid>, welche als legale Symbole genau jene Wörter zulässt, welche aus einer Folge von strikt aufsteigenden, gefolgt von einer Folge von strikt absteigenden Ziffern bestehen. Beispiele sind: 14, 121, 1221, 1341.
Sie dürfen annehmen, dass das leere Wort auch zugelassen wird. (Als Challenge können Sie probieren, das leere Wort auszuschliessen.)

Exercise 5: EBNF

2. Erstellen Sie eine Beschreibung $\langle \text{digitsum} \rangle$, welche als legale Symbole genau jene natürlichen Zahlen zulässt, deren Quersumme eine gerade Zahl ist.
3. Erstellen Sie eine Beschreibung $\langle \text{xyz} \rangle$, welche genau alle Wörter zulässt, die aus X, Y und Z bestehen und bei welchen jedes X mindestens ein Y im Teilwort links und rechts von sich hat. Beispiele sind: Z, YXY, YXXY, ZYXYY.
4. Erstellen Sie eine Beschreibung $\langle \text{term} \rangle$, welche als legale Symbole genau alle wohlgeformten arithmetischen Terme, bestehend aus positiven ganzen Zahlen, Variablen (x, y, z), Addition und Klammern zulässt. Geklammerte Terme müssen mindestens eine Addition enthalten. Beispiele sind: $1 + 4$, $(1 + 4)$, $1 + (3 + 4)$, $(1 + 1) + x + 5$.